# MODULE 3 – VALIDATION CONTROLS IN ASP.NET WEB FORMS

## 3.1 Learning Outcomes

**After this lesson, you should be able to:**

1. Explain what validation is and why it is needed in web forms.
2. Use common validation controls in ASP.NET Web Forms.
3. Understand `RegularExpressionValidator` for pattern-based checks.
4. Use `RangeValidator` and `CompareValidator` to validate numeric and other input.

ASP 3 | ASP.NET

### 3.2 What Is Validation?

**Validation** = checking if user input is acceptable before saving or processing it.

Examples:

- Is the field required? (not empty)
- Is the grade between 0 and 100?
- Does the email look like a valid email format?
- Are the two password fields the same?

**ASP.NET Web Forms gives you validation controls that:**

- Run both on client-side (JavaScript) when possible, and
- Run on server-side as a safety net.

### 3.3 Common Validation Controls

**Some important validation controls in Web Forms:**

- `RequiredFieldValidator` – checks that a control is not empty.
- `RegularExpressionValidator` – checks that input matches a pattern (regex).
- `RangeValidator` – checks that a value falls within a minimum and maximum.
- `CompareValidator` – compares the value of one control to another (or a fixed value).
- `CustomValidator` – lets you write your own custom logic in C#.

**All validation controls usually have:**

- `ControlToValidate` – the ID of the input control.
- `ErrorMessage` – the message to show when invalid.
- `Display` – how the error is shown (Static, Dynamic).
- `ForeColor` – color for the error text (often red).

### 3.4 Regular Expressions (Regex) – The Idea

A **regular expression (regex)** is a mini-language for describing text patterns.

Examples of patterns:

ASP 3 | ASP.NET

- Only digits
- Starts with “CTU-” then numbers
- Looks like an email address (something@something.com)

Regex feels complex at first, but you can start with simple ones.

Common pieces:

- ^ – start of string
- $ – end of string
- [0-9] – any digit
- [A-Za-z] – any letter
- {n} – repeat n times
- + – one or more
- * – zero or more

Examples:

- ^[0-9]{4}$
  - Exactly 4 digits (e.g., “2025”, “0001”).
- ^[A-Za-z]+$
  - One or more letters only.
- Simple email-like pattern (not perfect but OK for beginners):
  - ^[^@\s]+@[^@\s]+\.[^@\s]+$

### 3.5 RegularExpressionValidator

This control checks whether a TextBox value matches a regex pattern.

**Scenario:** Validate that a student ID is exactly 4 digits.

**Markup:**

```
<asp:TextBox ID="txtStudentId" runat="server">
</asp:TextBox>

<asp:RegularExpressionValidator
          ID="revStudentId"
```

ASP3 | ASP.NET

```
                runat="server"
                ControlToValidate="txtStudentId"
                ErrorMessage="Student ID must be exactly 4
                     digits."
                ValidationExpression="^[0-9]{4}$"
                ForeColor="Red">
</asp:RegularExpressionValidator>
```

- If user enters "1234" → valid.
- "12A4" or "12345" → invalid, error message shows.

No code-behind is required for this basic example; the validator works automatically.

### 3.6 RangeValidator

Validates that the value of a control falls between a minimum and maximum.

You must set:

- `MinimumValue`
- `MaximumValue`
- `Type (e.g., Integer, Double, Date, String)`

**Scenario:** Grade must be between 0 and 100.

```
<asp:TextBox ID="txtGrade" runat="server">
</asp:TextBox>

<asp:RangeValidator ID="rvGrade" runat="server"
     ControlToValidate="txtGrade"
     MinimumValue="0"
     MaximumValue="100"
     Type="Integer"
     ErrorMessage="Grade must be between 0 and 100."
     ForeColor="Red">
</asp:RangeValidator>
```

**Explanation:** If the user enters `-5` or `150` or `abc`, the validation fails.

### 3.7 CompareValidator

Compares one control's value with another, or with a fixed value.

Use it to:

- Check if a confirmation password matches the original.
- Ensure a date is after another date.

ASP.3 | ASP.NET

- Ensure an input is a valid number by comparing with type.

Key properties:

- `ControlToValidate`
- `ControlToCompare` OR `ValueToCompare`
- `Operator (Equal, NotEqual, GreaterThan, etc.)`
- `Type (String, Integer, Date, etc.)`

**Scenario 1: Confirm password**

```
<asp:TextBox ID="txtPassword" runat="server"
          TextMode="Password">
</asp:TextBox>
<br />

<asp:TextBox ID="txtConfirmPassword" runat="server"
          TextMode="Password">
</asp:TextBox>

<asp:CompareValidator ID="cvPassword" runat="server"
          ControlToValidate="txtConfirmPassword"
          ControlToCompare="txtPassword"
          Operator="Equal"
          Type="String"
          ErrorMessage="Passwords do not match."
          ForeColor="Red">
</asp:CompareValidator>
```

**Scenario 2: Ensure input is numeric**

```
<asp:TextBox ID="txtUnits" runat="server">
</asp:TextBox>

<asp:CompareValidator ID="cvUnits" runat="server"
          ControlToValidate="txtUnits"
          Operator="DataTypeCheck"
          Type="Integer"
          ErrorMessage="Please enter a whole number
          for units."
          ForeColor="Red">
</asp:CompareValidator>
```

### 3.8 Putting Validation Together – Enrollment Form Example

ASP3 | ASP.NET

**Goal:** Build a small form that validates:

- Name – required
- Email – required and must look like an email
- Age – between 16 and 60
- Password and Confirm Password – must match

Markup inside <form runat="server">:

```
<h3>Simple Enrollment Form</h3>

<!-- Name -->
<asp:Label ID="lblName" runat="server"
          Text="Full Name: ">
</asp:Label>

<asp:TextBox ID="txtName" runat="server">
</asp:TextBox>

<asp:RequiredFieldValidator
          ID="rfvName" runat="server"
          ControlToValidate="txtName"
          ErrorMessage="Name is required."
          ForeColor="Red">
</asp:RequiredFieldValidator>
<br /><br />

<!-- Email -->
<asp:Label ID="lblEmail" runat="server"
           Text="Email: ">
</asp:Label>
<asp:TextBox ID="txtEmail" runat="server">
</asp:TextBox>
<asp:RequiredFieldValidator
          ID="rfvEmail" runat="server"
          ControlToValidate="txtEmail"
          ErrorMessage="Email is required."
          ForeColor="Red">
</asp:RequiredFieldValidator>

<asp:RegularExpressionValidator
          ID="revEmail" runat="server"
          ControlToValidate="txtEmail"
          ErrorMessage="Please enter a valid email
          address."
```

ASP3 | ASP.NET

```
          ValidationExpression=
          "^[^@\s]+@[^@\s]+\.[^@\s]+$"
          ForeColor="Red">
</asp:RegularExpressionValidator>
<br /><br />

<!-- Age -->
<asp:Label ID="lblAge" runat="server"
          Text="Age: ">
</asp:Label>

<asp:TextBox ID="txtAge" runat="server"></asp:TextBox>

<asp:RangeValidator ID="rvAge" runat="server"
         ControlToValidate="txtAge"
         MinimumValue="16"
         MaximumValue="60"
         Type="Integer"
         ErrorMessage="Age must be between 16 and 60."
         ForeColor="Red">
</asp:RangeValidator>
<br /><br />

<!-- Password -->
<asp:Label ID="lblPassword" runat="server"
          Text="Password: ">
</asp:Label>

<asp:TextBox ID="txtPassword" runat="server"
          TextMode="Password">
</asp:TextBox>

<asp:RequiredFieldValidator
          ID="rfvPassword" runat="server"
          ControlToValidate="txtPassword"
          ErrorMessage="Password is required."
          ForeColor="Red">
</asp:RequiredFieldValidator>
<br /><br />

<!-- Confirm Password -->
<asp:Label ID="lblConfirm" runat="server"
          Text="Confirm Password: ">
</asp:Label>

<asp:TextBox ID="txtConfirm" runat="server"
          TextMode="Password">
```

ASP3 | ASP.NET

```
</asp:TextBox>

<asp:CompareValidator ID="cvPassword" runat="server"
          ControlToValidate="txtConfirm"
          ControlToCompare="txtPassword"
          Operator="Equal"
          Type="String"
          ErrorMessage="Passwords do not match."
          ForeColor="Red">
</asp:CompareValidator>
<br /><br />

<asp:Button ID="btnSubmit" runat="server"
          Text="Submit"
          OnClick="btnSubmit_Click" />
<br /><br />

<asp:Label ID="lblStatus" runat="server"></asp:Label>
```

**Code-behind (basic):**

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (Page.IsValid)
    {
       lblStatus.Text = "Form submitted successfully!";
    }
    else
    {
        lblStatus.Text = "Please correct the errors above.";
    }
}
```

`Page.IsValid` checks all validators on the page.

## 3.9 Workbook Activities (Validation)

### Activity V1 – Choose the Right Validator

For each requirement, write which validator(s) you would use:

1. "Field cannot be empty."
2. "Score must be between 0 and 50."
3. "Input must contain only digits (e.g., ID number)."
4. "Confirm email must be the same as the first email field."

ASP 3 | ASP.NET

---

**Activity V2 – Write a Simple Regex**

Write simple regular expressions for:

1. A 4-digit code (e.g., 1234).
2. A string of letters only (no digits, no spaces).
3. A student ID that starts with "CTU-" followed by 3 digits (e.g., CTU-101).

---

**Activity V3 – Design Your Own Validated Form**

Design (on paper, or in Visual Studio) a form called Registration.aspx with:

- Fields: Full Name, Email, Age, CourseCode (e.g., IT101).
- Validation rules:
  - Full Name: required
  - Email: required + email pattern
  - Age: between 15 and 70
  - CourseCode: must be 2–3 letters followed by 3 digits (e.g., IT101, CPE202)

Tasks:

1. Write which validators you will use for each field.
2. (Optional challenge) Write a regex pattern for the CourseCode requirement.

---

**Workbook Checkpoint – Validation Controls**

**Answer briefly:**

1. Why is it important to validate input both on the client and on the server?
2. Describe the role of ControlToValidate in a validation control.
3. When would you use a RangeValidator instead of a RegularExpressionValidator?
4. What does Page.IsValid do in the code-behind?

---

ASP 3 | ASP.NET